GeoRush – Jogo Sério para Ensinar Sólidos Geométricos

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	GR - GeoRush
Gerente de Projeto	Desenvolvedor Líder
Gerente/Responsável de GCS-DevOps	Desenvolvedor Líder
Fábrica de Software / Squad	Squad Voluntária Remota – 1 Desenvolvedor Líder + 4 Desenvolvedores Júniors
Repositório principal	https://github.com/GabBruno/GeoRush.git

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Gabriel Bruno

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente.)

Sumário 2
1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 4
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração 5
3. Ferramentas, Ambientes e Infraestrutura 5
3.1 Ferramentas do Pipeline 5
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 6
4. Identificação da Configuração 7
4.1 Itens de Configuração 7
4.2 Convenção de Nomenclatura 7
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) 7
5.1 Etapas do Pipeline de CI 8
5.2 Política de Pull Request e Revisão de Código 8
5.3 Critérios de Qualidade (Quality Gate) 8
6. Entrega e Implantação Contínua (CD) 8
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 8
6.3 Rollback e Recuperação 9
7. Infraestrutura como Código (IaC) 9
8. Controle de Configuração e Mudança 9
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 9
9. Baselines e Releases 9
9.1 Geração Automática de Baseline 9
9.2 Notas de Versão (Release Notes) 10
10. Estimativa do Status de Configuração 10
10.1 Monitoramento e Observabilidade 10
10.2 Métricas DORA 10
10.3 Relatórios e Auditorias 10
11. Marcos 10

12. Treinamento e Recursos 10
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 11
14. Quadro-Resumo do Pipeline CI/CD 11

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado à squad voluntária de 5 integrantes (1 Desenvolvedor Líder e 4 Desenvolvedores Juniores), responsável pelo desenvolvimento, versionamento e operação do jogo educacional "GeoRush". O plano abrange todo o ciclo de vida do código-fonte, desenvolvido na Godot Engine , assets gráficos/sonoros do jogo.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Project Charter – GeoRush (Termo de Abertura do Projeto).
- GDD (Game Design Document) do GeoRush v1.0.
- EAP e Dicionário da EAP do GeoRush.

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Garante que a Godot Engine exporte o projeto sem exposição de caminhos locais e monitora bibliotecas de terceiros (addons) usadas no jogo.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas sugeridas — substituir pelas efetivamente adotadas pelo projeto:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, IaC e pipelines como (build + testes verdes) código
Orquestração de CI/CD	GitHub Actions	Automação de build, testes, análise e implantação
Empacotamento	Export Templates nativos da Godot	Geração de arquivos HTML5/WebAssembly e .exe .
Registro de artefatos	GitHub Releases	Armazenamento dos arquivos `.zip` de cada versão do jogo

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Infraestrutura como Código	GitHub Pages	Hospedagem estática automatizada.
Monitoramento e Observabilidade	Google Analytics / GitHub Traffic	Métricas de acesso e estabilidade da página web.
Rastreamento de tarefas	GitHub Issues / Projetos	Vinculação de *commits* e *Pull Requests* às tarefas do jogo

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados via Infraestrutura como Código, garantindo paridade entre eles e permitindo recriação rápida e rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Integração contínua do código de desenvolvedor(es); recriado a cada pipeline.	Merge/push na branch principal (trunk)
Homologação / Staging	Testes de aceitação, integração entre serviços e validação funcional antes da produção.	Pipeline de CI aprovado (testes unitários e build sem erros na branch principal).
Produção	Ambiente utilizado pelos usuários finais.	Aprovação do gate de release (manual ou automática) após validação em staging

Por se tratar de um jogo executado no lado do cliente (Navegador via WebGL/WASM e tambem localmente atravas do executavel), a infraestrutura não exige *containers* complexos como Kubernetes ou Docker Swarm. O provisionamento focará em serviços de hospedagem de arquivos estáticos rápidos (CDN) para entregar os pacotes de *assets* aos alunos da escola sem latência e tambem para que ele possa baixar o executavel.

3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor…), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes)
- Arquivos de configuração por ambiente (variáveis, feature flags) — exceto chaves secretas, que devem ficar ambientes seguros.
- Documentação técnica (README, ADRs — Architecture Decision Records)

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<ticket>-<descrição-curta>	feature/SIQQ-123-login-oauth
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(auth): adiciona login via OAuth2
Tag de release	Versionamento Semântico vX.Y.Z	v2.3.0
...	...	...

4.3 Estrutura do Repositório

O repositório principal no GitHub seguirá a padronização oficial de projetos da Godot Engine, organizado da seguinte forma:

- `/.github/workflows/` (Scripts de automação CI/CD do GitHub Actions)
- `/assets/` (Imagens, texturas, spritesheets, sons e fontes)
- `/scenes/` (Cenas `.tscn` do Kratos, NPCs, prateleiras e UI)
- `/scripts/` (Códigos-fonte `.gd` / GDScript com a lógica do jogo)
- `/docs/` (GDD, Manuais e Project Charter)
- `project.godot` (Arquivo central de configuração da engine)

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (com cache).
2. Build da aplicação.
3. Análise estática de código (lint) e verificação de padrões de codificação.
4. Testes unitários, com medição de cobertura mínima.
5. Testes de integração.
6. Análise de segurança do código (SAST) e de dependências (SCA).
7. Empacotamento e publicação do artefato versionado no registro.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro desenvolvedor/colega antes do merge ("quatro olhos").
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate).

5.3 Critérios de Qualidade (Quality Gate)

- A exportação (Build) do jogo na Godot Engine deve ocorrer sem erros críticos de compilação ou arquivos faltantes.
- Zero *warnings* de dependências órfãs (arquivos `.tscn` ou *assets* excluídos que quebrem a cena).
- Nenhum PR (*Pull Request*) será aprovado se o jogo apresentar tela preta ou travamento no menu principal durante a validação automática.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

1. Deploy automático em ambiente de Homologação/Staging.
2. Execução de testes automatizados de aceitação/E2E em staging.
3. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
4. Deploy em produção com estratégia definida (ver 6.2).
5. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
6. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Substituição Estática	Os arquivos estáticos antigos da versão Web (HTML5/WASM) / Executável são substituídos integralmente pelos novos gerados pela Godot Engine, atualizando o jogo para todos os usuários instantaneamente.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. O pipeline de CD deve manter a versão anterior do Build (.zip) disponível para permitir rollback automatizado em caso de falha nos testes de fumaça ou nos indicadores de saúde pós-deploy.

7. Infraestrutura como Código (IaC)

Como a infraestrutura é baseada no GitHub Pages, o versionamento de infraestrutura resume-se às configurações do repositório (ex: rotas do GitHub Actions). Ambientes efêmeros não se aplicam à nossa topologia web estática.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

O Comitê de Controle de Mudanças (CCM) do GeoRush será composto pelo Desenvolvedor Líder, o Gerente do Projeto e o Professor Orientador (Sponsor). O CCM será acionado exclusivamente para:

1. Mudanças que alterem drasticamente o escopo do GDD, como exemplo: alterar as regras de progressão matemática das 5 fases.
2. Aprovação final para mover o jogo do ambiente de Homologação para Produção (lançamento oficial para os alunos).

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit e ao Build (.zip) publicado no GitHub Releases.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção devem ser monitorados continuamente (métricas, logs e alertas), permitindo identificar rapidamente falhas decorrentes de uma implantação.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

- **M1:** Estruturação do repositório no GitHub com proteção da *branch* `main` (bloqueio de *commits* diretos).
- **M2:** Implantação do Pipeline de CI (GitHub Actions rodando a Godot em nuvem e gerando a *build* Web sem erros).
- **M3:** Implantação do Pipeline de CD (Deploy automático da versão Web para um link de *Staging/Homologação*).
- **M4:** Release v1.0.0 (Lançamento da versão jogável automatizada em Produção).

12. Treinamento e Recursos

A equipe de 5 integrantes (1 Líder e 4 Juniores) necessitará de:

- Treinamento básico em versionamento de código com Git (criação de *branches*, abertura e revisão de *Pull Requests*, resolução de conflitos em arquivos da Godot).

- Capacitação específica para o Desenvolvedor Líder sobre como configurar *GitHub Actions* para exportar jogos da Godot 4 em modo *headless* (sem interface gráfica na nuvem).
- Padrões de *commits* convencionais (Conventional Commits) para geração automática de histórico (changelog).

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

O projeto utilizará bibliotecas e ferramentas de terceiros focadas no desenvolvimento do jogo .

- Todos os *addons* externos devem ser baixados e versionados diretamente na pasta `/addons/` do repositório para garantir que qualquer desenvolvedor consiga rodar o projeto localmente sem depender de downloads externos da *engine*.
- Nenhum *asset* pago ou código de terceiros com licenciamento restrito será incorporado sem a devida documentação e registro de licença no arquivo `README.md`.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de segurança, publicação do artefato/imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Staging	Deploy automático em homologação + testes de aceitação.	Merge na branch principal
Gate de Release	Aprovação para produção (automática ou manual, conforme criticidade).	Sucesso dos testes em staging
CD — Produção	Deploy via Substituição Estática (Overwrite) dos arquivos HTML5, smoke tests, geração de baseline/tag.	Aprovação do gate de release